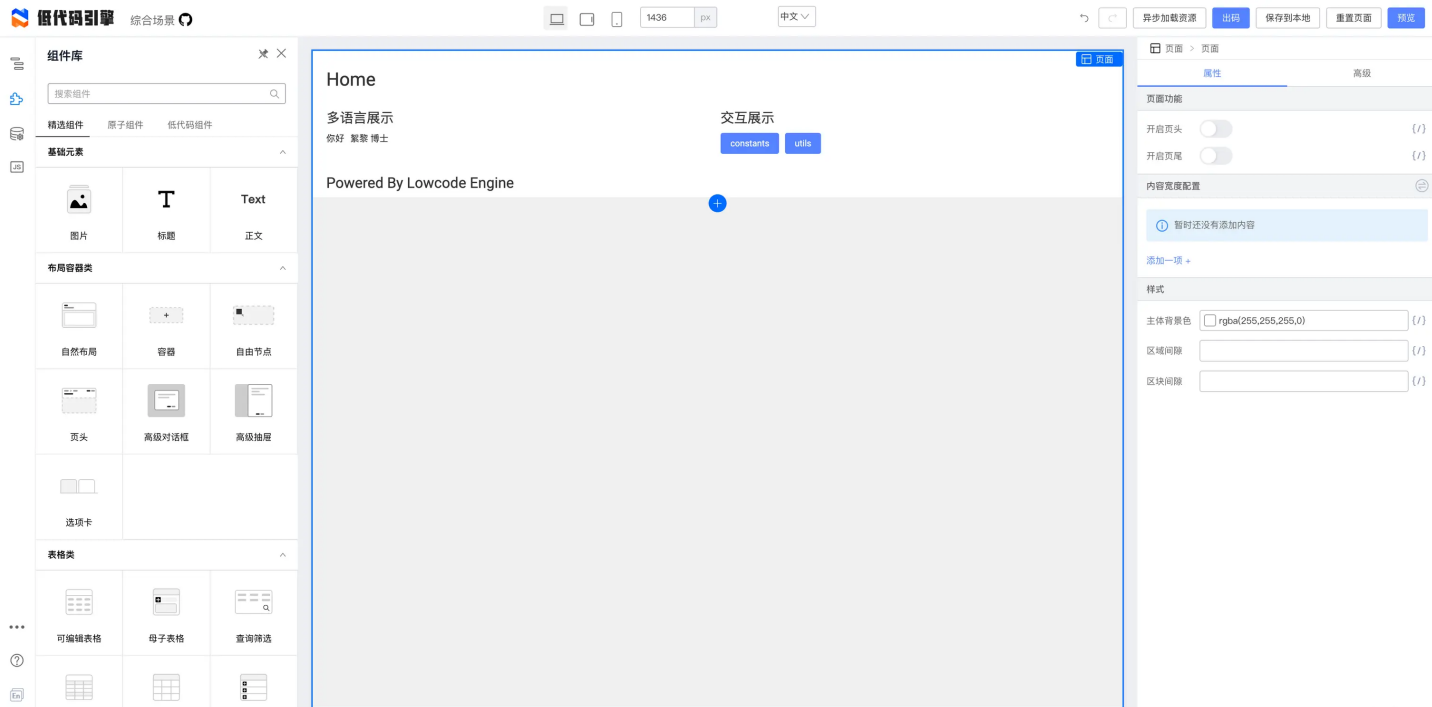


说说低代码细节含架构

关于低代码，想必大家都有所了解，就是用拖拉拽的方式来快速搭建某个页面 || 表单 || 海报 || 游戏等，但其实除了常见拖拉拽的方式以外，还可以用形如在线文档的方式来生成页面，这里分别贴个链接给大家体验一下：

当然了，本文主要讲解的是拖拽型的低代码，这里先简单截个图瞅瞅👉：



顺带罗列下低码平台的一些优缺点：

优点：✅

- 它的本质是提效，提效的同时给了自由度（以较少的成本达到想要的结果）
- 它的优势是可视和快速
- 它的能力源自于物料（组件）的能力

缺点：❌

- 上手成本（我自己觉得门槛还是高了），除了海报大部分情况下还是要摸索很久的
- 前端框架日新月异，过一两年来个改革，低码平台就得适配一遍，维护成本相当大；又或者平台自身升级了，你也不知道会对自己的项目产生什么影响，但是不跟着升级后面就没法迭代；并且基本上你用了平台的低代码，要想切换到其他平台那就得从 0 到 1
- 不好维护和迭代，换个同类型组件都得考虑一下，如果换个人还得重新理解，还不好调试，不好定位问题，也不好确定本次改动对之前功能的影响

- 不好优化（比如你现在想做性能优化，完全没有头绪🤯）
- 二次开发不可逆，很难做到持续可视化（确实，大部分情况是这样👩‍🔧♀️）
- 随便举个反例，只要低代码平台没有实现那就是缺点😁
- ...

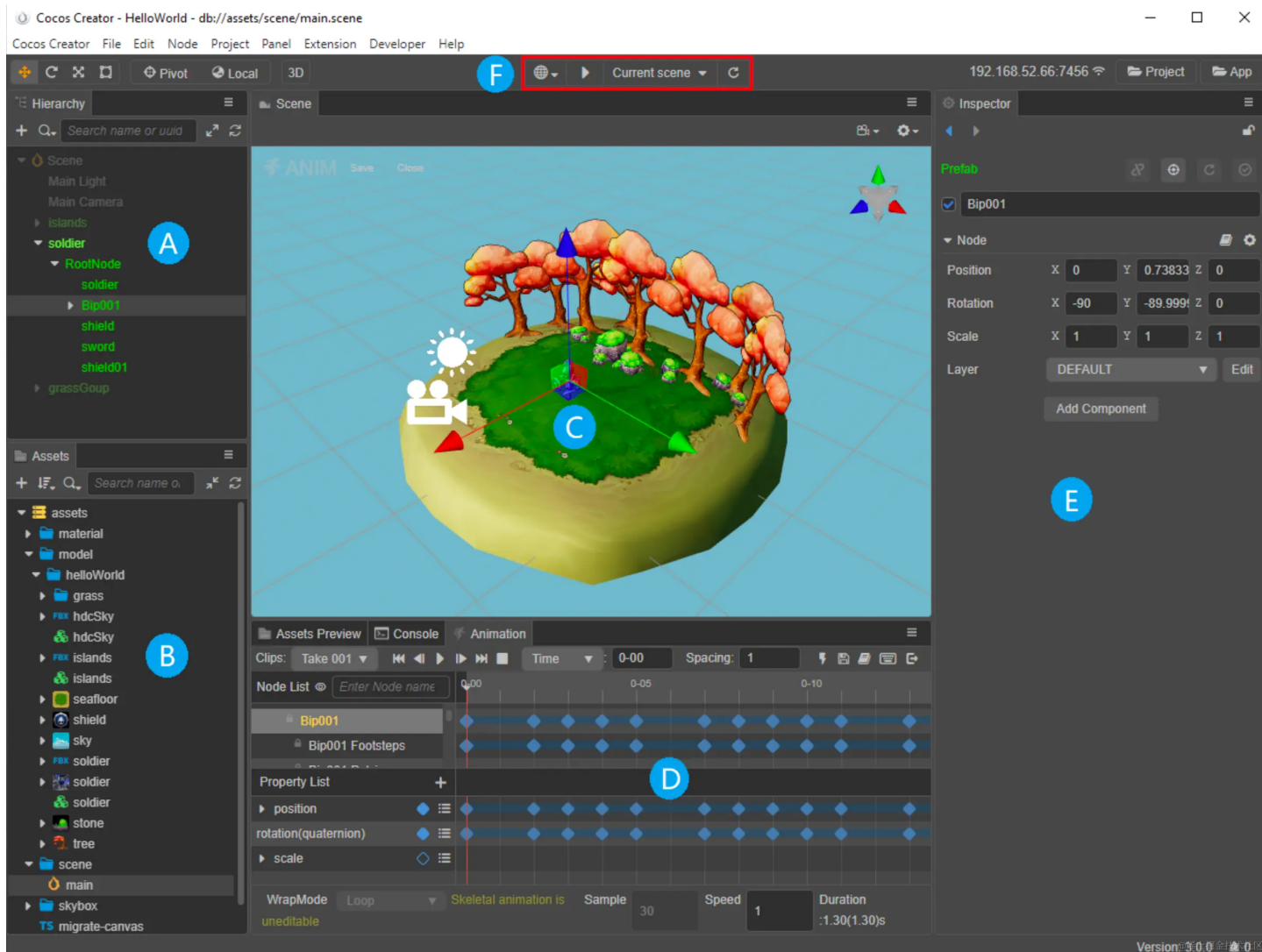
确实，低代码自身限制还是很多的，随便提个改动，都可能牵一发动全身。不过这里我们并不会去评判低码平台的好坏，而只是单纯的分享一些低代码中的核心思路和整个流程：包括但不限于模块划分、如何解耦和扩展、画布的实现方式等等。

最重要的事

最重要

如果让我说一个低代码中最重要的事情，那就是方向👉。我们要知道低代码是有它的适用场景的（交互简单 && 轻逻辑），比如：

- 海报（我觉得这个方向应用的相当好，0 逻辑 0 交互）
- H5 运营活动页（一次性的页面也很适用，没有维护的烦恼）
- 表单收集页（问卷调查也不错，也是一次性）
- 中后台页面
- [2D 游戏](#)（其实我觉得很多低代码思想是从游戏引擎借鉴过来的，因为大家可能比较少触碰，所以这里特地截了一个游戏开发时候的图，和低代码一毛一样）

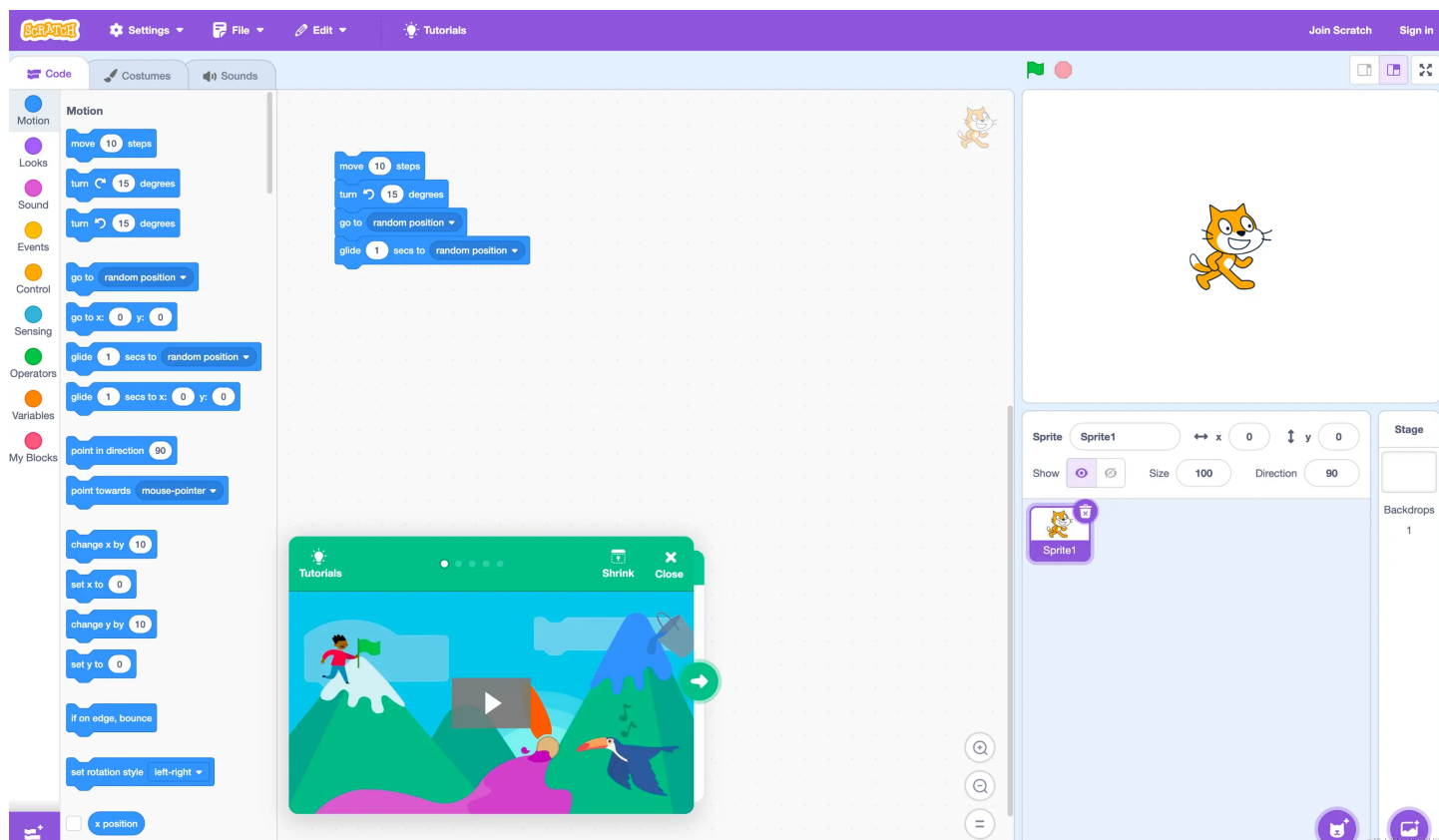


单单只实现上面的任意一种场景就已经要兼容很多东西了，所以想覆盖所有情况几乎是不可能的，而且现在也没有一个统一的规范，都是各做各的，百花齐放。因此不要想着啥都做🐼，得挑一个垂直方向发力，方向越具体，自动化程度越高（比如组件拿来即用，无需修改），生产效率也更高，平台也会更好用。

次重要

除了方向，次重要的事情就是简单（包括交互和逻辑）。为什么要简单呢？

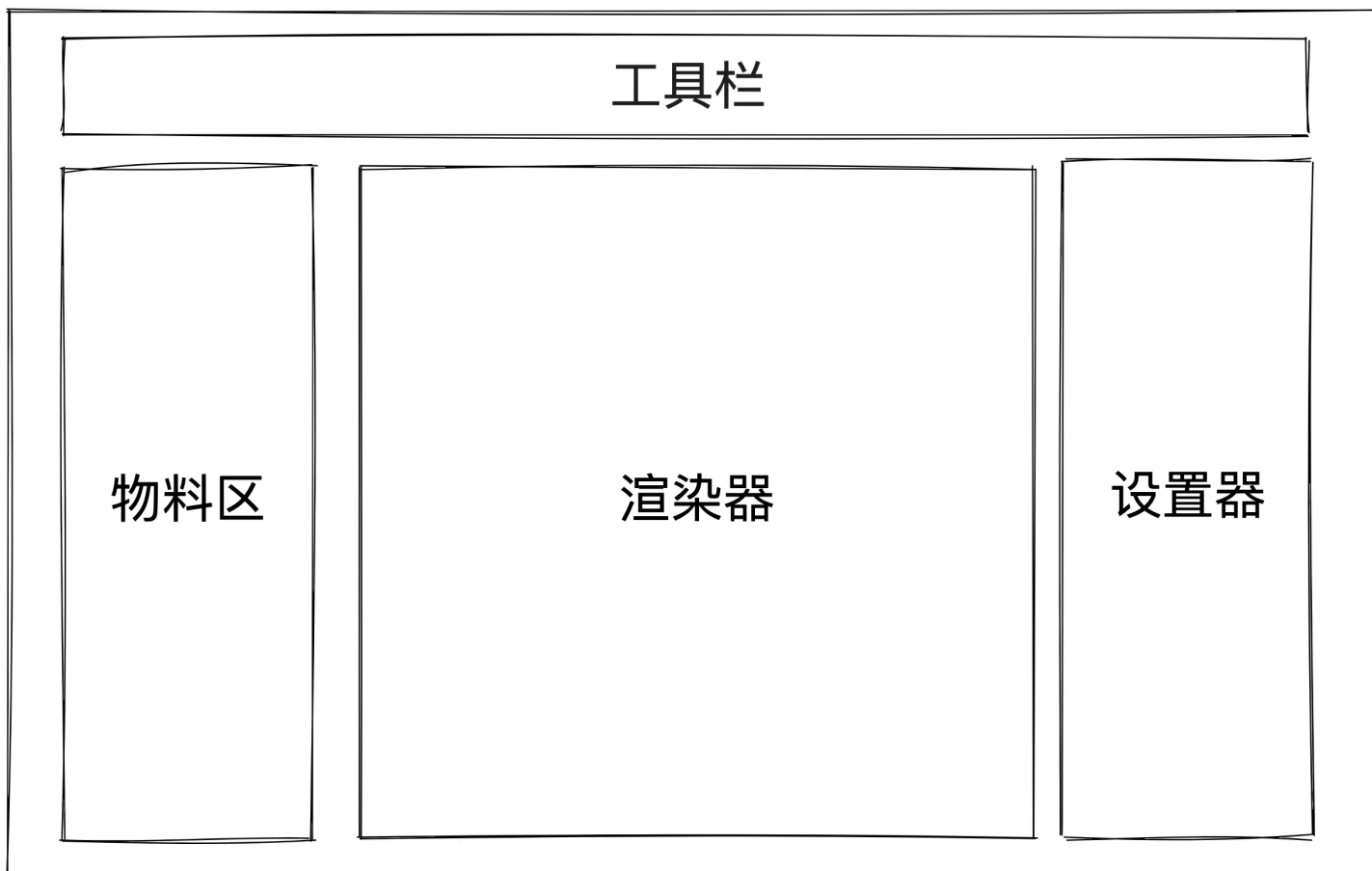
- 因为一旦复杂就不可视了，本来拖拽的优势也没有了
- 通常情况下，低码平台只实现了视图可视化，并没有实现逻辑可视化，逻辑一复杂还是得写代码。根本原因是我们很难将逻辑进行可视化，它不如写代码来的干脆明了。当然目前也有比较适合于逻辑可视化的场景（前提是逻辑比较固定），比如审批流和Scratch，也都特地截了个图👉：



那如何才能做到简单呢？还是得朝着垂直方向发力😓，也就是会有点定制，要固化一些操作、约束一些行为。目前我还是觉得低代码主要还是给非研发同学用的，所以要足够简单😓。

基本实现

扯了这么多，现在让我们赶紧步入正轨吧😁。纵观大部分低码平台，主要都是由以下四部分组成（画的有点简陋）：



接下来我们会对每个部分都挑两三个重点来讲解一下🚗。

1、协议

在讲解每个模块之前，我们先来说一个东西，就是协议（听起来很高大上，其实就是规范，更朴素点叫做格式），它主要包括物料协议、平台搭建协议和其他协议等等。为什么要先约定协议呢，因为这个东西贯穿低代码的整条链路：

- 当我们想扩展物料时，需要实现相应的协议
- 当我们把左侧一个物料拖到中间画布区时，需要通过协议来通信和解析
- 当我们选中画布区域的组件时，要想通过右边设置面板进行属性设置时，也需要通过协议来通信和解析
- 当我们准备预览和发布代码时，也需要通过协议来生成代码

协议是低码平台的基石，它的主要目的就是约束和扩展，约定的好，事半功倍；约定不好，版版重构😂。约定优于配置说的就是这个道理。如果维护到后期发现协议很难拓展了，那基本只能重来了，只不过你多了些经验。协议本质上就是一堆 interface（就是固定格式啦）。

2、物料区

先来看看最左侧的物料区吧，这是低代码的起点，协议也是从这边开始的，先约定个最简单的物料协议吧：

```
1 /** 单个物料约定 */interface IComponent { /** 组件名 */ componentName: string;
```

```

/** 组件中文名称 */ title: string; /** 缩略图 */ icon?: string; /** 包地址 */
npm: { /** 源码组件名称 */ componentName?: string; /** 源码组件库名 */
package: string; /** 源码组件版本号 */ version?: string; }; /** 分类: 比
如基础组件、容器组件、自定义组件 */ group?: string; /** 组件入参或者说是可配置参数 */
props?: { name: string, propTypes: string, description: string,
defaultValue: any, }[]; /** 其他扩展协议 */ [key: string]: any; } // 举个例子
const componentList = [ { componentName: "Message", title: "Message",
icon: "", group: "基础组件", npm: { // import { Message } from
@alifd/next 的意思 exportName: "Message", package: "@alifd/next",
version: "1.19.18", main: "src/index.js", destructuring: true,
}, props: [{ name: "title", propTypes: "string", description:
"标题", defaultValue: "标题" }] }];

```

物料区其实没啥功能，我们会会有一个 componentList，里面是各种组件的基本信息，直接循环渲染即可。同时还会顺便生成一个 componentMap，主要是方便后续我们通过组件名来快速获取是组件的元信息，比如下面这样👉：

```

1 const componentMap = { Message: { componentName: "Message",
title: "Message", icon: "", group: "基础组件", // ...
},}; // 通常情况，低码平台平台还需要对外暴露出加载组件和注册组件的方法，比如这样👉：
function createRegisterConfig() { const componentList = []; const
componentMap = {}; return { componentList, componentMap,
loadComponent: () => {}, register: (comp) => {
componentList.push(comp); componentMap[comp.componentName] = comp;
} }}

```

物料区本身并不复杂，这里就说三个注意点：

- 组件的分类：
 - 容器组件（这类组件主要用来协助布局和嵌套）
 - 基本组件（常见的有图片、文本、输入框、视频等）
 - 集成度稍高一点的组件（表格、表单、图表、自定义组件、业务组件）
- 如何加载组件？不管什么情况下，在前端加载文件只有两种方式：
 - 一个是 `import()`
 - 一个是 `<script>`
- 这里也简单贴个示例代码：

```

1 // 方法一: import
const name = 'Button' // 组件名称
const component = await
import('https://xxx.xxx/bundle.js')
Vue.component(name, component) // 方法二:

```

```
scriptfunction loadjs(url) {    return new Promise((resolve, reject) => {  
    const script = document.createElement('script')    script.src = url  
    script.onload = resolve    script.onerror = reject    })}const name =  
'Button' // 组件名称await loadjs('https://xxx.xxx/bundle.js')// 这种方式加载组件，  
会直接将组件挂载在全局变量 window 下，所以 window[name] 取值后就是组件  
Vue.component(name, window[name])
```

• 如何扩展自定义组件？

- 这是最为重要的功能之一，我们要想让一个第三方组件在当前平台可用，直接拿来肯定是不行的，必须要进行适配：
 - 如果这个第三方组件是已有的，我们需要简单适配一下（适配器模式），也就是这个第三方组件需要实现 IComponent 这个接口
 - 如果这个第三方组件是待开发的，那低码平台一般会提供脚手架让使用方去基于一个固定的模板去开发，这样协议自然也就对上了
- 通常情况下，低码平台自身会有个物料管理平台对组件进行统一的管理和操作，简单点做的话我们可以直接把开发好的组件发到 npm 上，把 npm 当做物料平台用
- 但我们更期望的是物料统一，既然用了别人的平台，那就去用别人的组件，尽量去复用平台自身已有的组件库。我们不生产组件（或者少生产），只是消费组件 😊。

3、画布区

这里我们先说说画布区的几种实现方式吧：

- 自由画布：拖到哪里元素就放到哪里，配合容器组件可以协助布局以实现自适应
- 流式布局：组件从左到右从上往下自然排列，比如 H5 就是单纯的从上往下平铺开
- 自动布局：也是拖哪放哪，但是原来的地方如果已经有组件则会被挤开，被挤开的元素还会有个递归挤开的过程
- 栅格画布：类似前端组件库中的栅格布局，一行 24 列，一行 12 列这样划分填充，看起来比较规范也比较整齐
- 网格画布：类似棋盘一样的网格，每个网格都是 8px（看 UI 规范），拖拽组件的时候会自动吸附到这些网格线上，这样可以让整体看起来更加错落有致（如果你有一些设计规范，可能会需要用到这种画布）
- 混合布局：效率和通用性的权衡

那画布区是怎么渲染出来的呢？其实我们会会有一个 componentTree 来递归渲染拖拽出来的元素，然后各自按照组件类型来渲染，先简单看下 componentTree 的大体结构吧 👉：

```
1 import ReactRenderer from '@alilc/lowcode-react-renderer';import ReactDOM from  
  'react-dom';import { Button } from '@alifd/next';const componentTree = { // 画布
```

```

区的所有元素都在这里维护  componentName: 'Page', // 因为我们是页面为单位, 所以顶层一
定是 Page props: {}, children: [{ componentName: 'Button', props: {
  type: 'primary', size: "large", style: { color: '#2077ff'
  }, className: 'custom-button', }, children: '确定', }]};const
components = { Button,};ReactDOM.render(( // 里面会递归渲染组件
<ReactRenderer componentTree={componentTree} components={components} />
), document.getElementById('root'));//
=====// 如果树形结构不好理解的话, 我们可
以把数据换成普通的数组来理解, 然后渲染的过程就是循环遍历数组即可, 这样就容易多了, 比如 H5
页面就很适合数组这种形式const componentTree = [{ "componentName": "ElButton",
  "height": 100, "props": {}, "style": {}}, { "componentName":
"ElInput", "height": 300, "props": {}, "style": {}}];

```

事实上, 低码平台都秉持着数据驱动视图的思想 (和我们现在用的 vue 和 react 框架如出一辙), 也是通过递归解析 componentTree 这个全局组件树来动态生成页面, 化简来说就是: `UI = Transformer(componentTree)`。Transformer 这一步我们可以称之为转换器、渲染器或者 render 函数, 通常开发完成之后, 这个渲染器是不用改的, 我们只需要单纯的修改数据, 渲染器自然会帮我们解析。

这个思想很重要, 也是解耦的核心: 就是我们所有的操作, 不管拖拽也好, 修改元素属性也好, 还是调整元素位置, 都是对 componentTree 这个数据进行修改, 单纯的对数据进行操作, 比如追加元素, 就往 componentTree 的 children 里面 push 一个元素即可; 如果要修改一个元素的属性值, 只需要找到对应元素的数据修改其 props 值即可。画布编排的本质就是操作组件节点和属性。

另外为了让每个组件都能直接获取到这个 componentTree, 我们可以把这个 componentTree 弄成全局的 (全局数据能够让整体流程更加清晰), 比如放在 window、vuex、redux 上, 这样每个模块就能共享同一份数据, 也能随时随地更改同一份数据 (平台会暴露公共的修改方法), 而这个渲染器只是单纯的根据这个数据来渲染, 并不处理其他事情。

这里我们还要注意一个问题, 就是画布区本身也是个组件, 是个组件那它就会受到父元素和全局的影响, 最简单的比如样式, 可能受到外部样式作用, 导致你这个画布区和最终呈现的页面可能有一丢丢的不同, 所以要排除这些影响, 那具体可以怎么做呢😞? 就是把这个画布区搞成一个独立的 iframe, 这样环境就比较纯了, 完美隔离, 只不过增加了通信的成本。现在, 物料区只负责渲染组件列表, 以及触发拖拽放下的事件, 之后就是触发修改全局的 componentTree, 再之后就是触发画布区的重新渲染。这样一来, 你就会发现画布区和物料区就很好的解耦了。到目前为止画布区只负责单纯的渲染。

如果你还是想知道这个渲染器到底是怎么递归怎么渲染的, 我这里也提供了段简单的代码帮助你理解:

```

1 function renderNode(node) { if (!node) return null; const component =
  components[node.componentName]; const props = compute(node.props); const
  children = node.children.map(c => renderNode(c)); return
  React.render(component, props, children);renderNode( componentTree);

```

4、属性设置区

接下来我们简单讲下右侧的属性设置区，这个区域通常会支持三种基本的设置：props && 样式 && 事件。一般来说我们的操作是这样的：

- 点选画布区的某个组件
- 触发设置某个全局变量为当前组件
- 右侧属性面板就会根据当前组件的 `componentName` 从 `componentMap` 中找到组件对应的 `setters`（可配置项），这个 `setters` 其实就是一开始在物料协议里面约定的 `props`，但是和 `props` 可能有点小区别，需要自己手动写个函数转一下；或者可以直接在物料协议里面多添加一个 `setters` 字段来专门描述有哪些属性可以支持配置。两种方式都是可以的
- 然后也是单纯的循环渲染 `setters`
- 再把当前组件的 `state`（初始值）赋值给 `setters`

上面的过程和我们平时开发中后台应用的表单项（`FormRender`）是一毛一样的，网上也有很多教程，这里就不细说了。

我们主要来讲一下属性设置区的几个注意点：

- 首先如果我们修改了属性设置区的表单项，我们实际上是去修改全局的 `componentTree`，然后画布区自然就会根据这个新的 `componentTree` 自动渲染，有点单向数据流的意思（就是修改数据的入口只有一个），也方便排查问题。把数据放到全局上，很多通信的过程就可以省掉了
- 一个常常提到的问题就是如何实现联动，比如字段 2 的显隐依赖于字段 1 的值，类似这种功能通常有两种实现方式：
 - 一种类似发布订阅，我们可以在字段 2 中监听（`on`）来自字段 1 的 `emit` 事件，只是多了你就很难知道各自有哪些依赖关系了
 - 另一种方式就是利用全局数据了，比如我们把字段 1 和字段 2 都放在全局数据中，然后在字段 2 中新增一个 `visible` 的属性设置器，其值是一个模板表达式，形如：`{{ globalData.field1 && ... && globalData.fieldN }}`，因为数据是全局的所以很方便能够直接获取到，在实际渲染的过程中就会动态执行上面那个表达式来确定组件渲不渲染。此外，因为数据是全局的，跨组件或者跨页面共享数据也会变得轻而易举
- 再一个常常提到的问题就是如何处理点击事件？如果做的开放点、简单点，我们可以直接让用户自己写函数，然后运行的时候用 `eval` 或者 `new Function` 执行一下就行。但是这样会有个问题，就是安全性、稳定性和效率不够，所以我们需要进行一些限制，这个通常有两种方法：
 - 一种是暴露固定方法，只接收参数，比如我这个点击的结果就是跳转到某个页面，也就是执行 `window.open` 这个方法，那我们就不允许用户直接书写这个代码，而是先内置一个全局封装好的 `jumpToPage(url)` 方法，然后在属性设置的时候只允许输入 `url` 并进行简单校验
 - 但是固定方法是很难满足我们的一些需求的，最终还是得支持让用户可以自己写脚本，于是乎我们就得让这个脚本具有良好的隔离性，也就是沙箱或者对代码进行校验等，这里就简单说一下沙箱的方式：

- with + new Function (用 with 改变作用域实现隔离、用 try catch 捕获错误保障稳定性)
- iframe 沙箱: 在一个空的 iframe 里面执行这些未知的代码可以最大程度的实现隔离, 其余方式都可以通过原型链进行逃逸 (其实 iframe 通过 parent 也能逃逸😂)
- 等一个新的 API: ShadowRealm

◦ 下面是简单的代码示例截图, 有个印象就行:

```
export function parseExpression(str: any, self: any, thisRequired = false) {
  try {
    const contextArr = ['use strict;', 'var __self = arguments[0];'];
    contextArr.push('return ');
    let tarStr: string;

    tarStr = (str.value || '').trim();

    // NOTE: use __self replace 'this' in the original function str
    // may be wrong in extreme case which contains '__self' already
    tarStr = tarStr.replace(/this(\W|$)/g, (_a: any, b: any) => `__self${b}`);
    tarStr = contextArr.join('\n') + tarStr;

    // 默认调用顶层窗口的parseObj, 保障new Function的window对象是顶层的window对象
    if (inSameDomain() && (window.parent as any).__newFunc) {
      return (window.parent as any).__newFunc(tarStr)(self);
    }
    const code = `with(${thisRequired ? '{}' : '$scope || {}'}) { ${tarStr} }`;
    return new Function('$scope', code)(self);
  } catch (err) {
    logger.error('parseExpression.error', err, str, self?.__self ?? self);
    return undefined;
  }
}
```

ts 复制代码

```
1 private executeCode(code: string, globalScope: InjectVMVarsType) {
2   const sandbox = this.iframe.contentWindow as IBrowserRuntimeVMWindow
3   sandbox.__INJECT_VARS__ = globalScope;
4
5   return sandbox.eval(`
6     (() => {
7       with (window.__INJECT_VARS__) {
8         return (${code})
9       }
10    })()
11  `);
12 }
```


那沙箱里面能拿到什么数据呢？其实主要看我们想暴露什么参数给组件了，比如全局数据、全局方法、父元素、当前组件的 state 等等，那其实我们就可以直接传个包含以上数据的大对象然后传给 with 即可。

5、顶部操作区

上面那几个组成部分其实已经构成了低代码中最核心的几个部分，我们可以称之为 Core（内核）。对于顶部操作区，通常可以有前进、后退、清空等操作，但是这些操作并不算是必须的，它们通常以插件的形式存在，也方便大家一起维护和扩展，这就是微内核架构：

1 * Core + N * plugins

然后我们这里就简单讲下清空和回退操作吧（当然其余其他操作也是一样的道理）：

- 先说下清空，这个其实就很简单了，就是直接把全局的数据清了就行，画布区会因为数据变了而自动重新渲染（注意：我们做的任何操作都是先去修改那个全局数据）。
- 再说说回退吧，对于大部分编辑器来说，这是一个很常见的功能。
 - 简单做的话就是每次有操作时，直接把整个 componentTree 复制一份，撤销和恢复都直接重新赋值整个 componentTree 即可
 - 另外一种方法就是基于操作来回退，源数据只有一份，但是有多个 actions，比如我添加了一个元素，对应 `add(comp)` 方法，那我回退的时候就是执行它的反向操作 `remove(comp)` 来修改，麻烦的地方就在于每个操作都需要写一个对应的反向操作的方法

但其实插件不仅仅适用于顶部条：

- 物料区也可以有插件：主要表现就是用脚手架来扩展物料
- 画布区也可以有插件：比如选中组件的时候可以扩展复制、删除、辅助线、上移下移等功能
- 设置区也可以有插件：比如颜色选择器，可以追加自定义表单项

6、代码生成

到目前为止，我们就初步搭建好了页面，它背后其实是一堆数据，但是有可能你还是一头雾水，没关系，为了能让大家有个更具象的认识，我把这份数据的最终形态简单汇总了下（放心，就一丢丢丢丢数据☑️很好看懂的）：

```
1 const json = { version: "1.0.0", // 当前协议版本号 componentList: [ // 组件
  // 描述
  { componentName: "Button", package: "@alifd/next", version: "1.0.0", destructuring: true },
  { componentName: "Page", package: "@alifd/next", version: "1.3.2", destructuring: true },
], state: { // 全局状态 name: "尤水就下" }, componentsTree: { // 画布内容
  componentName: "Page", props: {}, children: [{ componentName: "Button", state: {}, props: { type: 'primary', size: 'large', style: { color: 'red' }, className: 'custom-button', onClick: { // 事件绑定 type: "JSFunction",
```

```
value: "function(e) { console.log(e.target.innerText) }" },  
}] }
```

接下来就是要准备发布了，具体该怎么做呢？这里说下两种主要的方式：

- 同技术栈：如果用的技术栈和低代码平台是一样的话，我们可以使用运行时渲染，开发一个 preview 的页面，页面里面有固定的渲染器（和画布区有点像），也会加载对应的组件库，剩下的就是远程获取页面的 json 数据（也就是上面那一坨代码，当然也可以称之为 schema），传递给渲染器，然后直接展示即可。不理解？那就再简单贴下代码：

```
1 import React, { memo } from 'react';import ReactRenderer from '@alilc/lowcode-  
react-renderer';const SamplePreview = memo(() => { return ( // 至于渲染器怎么实现的，网上有一堆文章，不理解的同样可以把数据当成简单的数组，for 循环渲染而已  
<ReactRenderer      className="lowcode-plugin-sample-preview-content"  
schema={schema}      components={components}    /> );});// ps: 这里简单说下  
json 和 schema 的区别，以前我也很困惑😂，不知道现在理解对没有（但我感觉大部分情况下都是随便叫的）// json 是一个普通对象// schema 是一个有固定格式的对象// json schema 是一个描述 json 格式的普通对象
```

- 另一种就是直接导出项目，在此基础上可以二次开发或者尝试 ssr 渲染（但是一旦二次开发基本就是不可逆罗），就像下面这样👉：

那两种方式有什么区别呢：

😂	前者	后者
1	运行时编译 (JIT)	预编译 (AOT)
2	最终输出的是 json	最终输出的是项目
3	实时生效	得打包重新发版
4	一般这个够用	有性能要求用这个

为什么前者会有性能问题呢😞？因为渲染页面的时候，需要将 json 进行一层转换，而后者是已经打包之后的项目了。

其他问题

- 模块之间如何进行解耦呢：
 - 通过全局的发布订阅模式来通信

- 通过全局数据来共享，而不是通过直接相互传递数据
- 其实你把每个模块都写成独立的项目，开发起来自然就会强迫自己解耦了
- 解耦最大的好处就是当你要重构或者替换某一个模块时，可以直接替换单个模块而不影响其他地方的逻辑
- 除了全局变量，我能在组件里面维护自己的状态吗？当然是可以的，我们可以新增一个 state 属性，来维护组件自身的一些状态，并且我们可以通过维护原型链的方式来层层向上获取父元素数据，比如这样：`state.__proto__ = parent.state`
- 怎么支持跨平台：低代码平台搭建好后最终得到的是一个 json，而这个 json 本身就是以一种通用的语言来描述页面结构、表现和行为的，它和平台无关。要想跨平台、跨框架，只需要做一个适配层去解析这些 json，导出成各平台、各框架需要的样子即可。举个例子来说，我们在解析的过程中肯定会需要创建元素，而 vue 和 react 都有各自的 createElement 方法，那到时候只需要把创建元素的方法换成各框架各自的方法就行了。思路听起来很简单，但是每一种适配起来都很繁琐
- 在哪里管理依赖呢：低码平台除了上面提到的模块，一般还会有个物料管理平台，它不仅用来管理我们的组件，也会管理我们的项目依赖，最终表现为 json 中的 packages 字段，就像下面这样：

```
1 {"version": "1.0.0", "packages": [{ // 依赖包    "title": "fusion 组件库",  
    "package": "@alifd/next",    "version": "1.20.0",    "urls": [  
    "https://alifd.alicdn.com/npm/@alifd/next/1.20.0/next.min.js",  
    "https://alifd.alicdn.com/npm/@alifd/next/1.20.0/next.min.css"    ],  
    "library": "Next" // 最终这样用: window.Next.componentName}], "components": []}
```

- 辅助线的实现（测距、对齐、吸附）：需要遍历所有物体拿到对应的 x、y 值，只不过在遍历较多组件的时候可以有些策略，比如是否在可视区、和目标元素的距离、超过十条或者 5ms 就停止遍历等等
- 版本维护：低码平台通常会有 version 字段来维护每个迭代，并且会有好多种 version，包括但不限于：协议有版本号、依赖有版本号、组件有版本号、每次发布也有版本号
- 怎么调用接口：通常会有一个单独的模块来配置所有接口，然后再在右侧的属性设置面板中就挑选对应的接口即可
- 如何监控和埋点：接个监控和埋点库，并在全局暴露方法，方便用户在右侧属性面板添加自定义事件
- 多人开发同一个页面如何解决冲突？假如我们有两个人在编辑同一个页面，先后点了保存该怎么处理？其实不管你怎么编辑，最终保存的就只有 json 数据，那么我们就可以把问题转换为怎么解决两个不同 json 的冲突，有两种方式：
 - 在项目生成之初，会建个 gitlab 仓库，借助 git 实现基于行的文本对比，和我们平时开发一样，只不过这里只有一个 json 文件

- 两个 json 合并有冲突根本原因是某个字段或者某个属性值变了，所以可以自行实现一套对象合并策略，把 json1 和 json2 的不同之处罗列出来，让用户保存的时候手动选择要保留哪一个
- 怎么调试？如果你是前端或许看看控制台报错还能知道是什么问题，但如果你是非研发同学，那就完全没招了。对此，就需要低码平台开发一个调试模块，形如 vue 和 react 的 devtool 和 vConsole 调试面板，当然这对非研发同学还是不够友好，我们最好需要将具体的错误和建议在页面上醒目的展示出来
- D2C && AI+：低码平台需要我们自己搭建页面，D2C 则又省了一步，直接解析设计稿，经过繁琐的转换后转成了我们的 json，然后就直接生成了页面。可行是可行，现在也有很多这样的产品，并且结合 AI 能够为其增色不少。但是吧，我觉得这玩意几乎维护不下去，不过画饼需要 😊

小结

如果你能看到这里，那说..明...文章写的还行，哈哈哈哈哈 🤪。这里我就简单的用几句话对本篇文章进行一个总结：

- 低代码最重要的就是：方向
- 低代码的基石就是：协议
- 低代码的最大的优势：可视和高效
- 低代码所有的操作都是在操作一个 json

最后的最后，我又重新画了张图方便大家记忆 🙌：

